Day 18: ConfigMaps, Secrets & Namespaces

COVER SLIDE / OVERVIEW

ubuntu@master-node-dhiraj:~/day18-configmaps

```
# Namespace Isolation, ConfigMaps, Secrets & Volumes
$ kubectl create namespace dev && kubectl create namespace prod
$ kubectl apply -f configmap-dev.yaml -n dev
$ kubectl create secret generic db-secret --from-literal=password=dev-
pass -n dev
* Status: Workloads & Volume Mounts Verified
```

MISSION ROADMAP (01 TO 07)

- ✓ **01: Isolated Namespaces Setup (dev & prod)**
- ✓ **02: Environment-Specific ConfigMaps**
- ✓ **03: Lab-Only Base64 Secret Management**
- ✓ **04: Multi-Environment Deployment & Mounts**
- ✓ **05: Port-Forwarding & Environment Verification**
- ✓ **06: Config Update & Rollout Restart Behavior**
- ✓ **07: Interview Readiness Q&A**

01 / Core Concepts & Configuration Architecture

Kubernetes Configuration Concepts

- **Namespaces:** Provide logical isolation within the same physical cluster for dev, test, and prod workloads.
- **ConfigMaps:** Store non-sensitive key-value configuration data decoupled from container images.
- **Secrets:** Store sensitive data (passwords, tokens) encoded in base64 (note: encoding is not encryption).
- **Environment Injection:** Exposes configuration values into the container runtime via `env` and `valueFrom`.
- **Volume Mounts:** Mounts ConfigMap data as configuration files directly into container directory paths.
- **Update Behavior:** Mounted volume files sync automatically after a short delay, whereas environment variables require a pod restart.

Essential kubectl Commands

- `kubectl create namespace <name>`
- `kubectl create secret generic <name> --from-literal=key=val -n <ns>`
- `kubectl apply -f deployment.yaml -n <ns>`
- `kubectl port-forward -n <ns> deploy/web <port>:80`
- `kubectl exec -n <ns> deploy/web -- printenv APP_ENV`
- `kubectl rollout restart deployment/web -n <ns>`

02 / Isolated Namespaces Creation (Task A)

Task A: Created dedicated `dev` and `prod` namespaces to establish strict environment boundaries and prevent resource naming collisions across stages.

```
ubuntu@master-node-dhiraj:~$ kubectl create namespace dev
namespace/dev created
ubuntu@master-node-dhiraj:~$ kubectl create namespace prod
namespace/prod created
ubuntu@master-node-dhiraj:~$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   4d23h
dev               Active   29s
kube-flannel      Active   4d23h
kube-node-lease   Active   4d23h
kube-public       Active   4d23h
kube-system       Active   4d23h
prod              Active   10s
```

Namespace Verification Summary

- Successfully executed `kubectl create namespace dev` and `prod`.
- Verified active status and existence using `kubectl get namespaces`.
- Established clean scoping boundaries for upcoming ConfigMaps, Secrets, and Deployments.

03 / Environment-Specific ConfigMaps (Task B)

Task B: Created environment-tailored ConfigMaps (`site`) containing `APP_ENV` values and embedded `index.html` web templates for both `dev` and `prod` namespaces.

```
ubuntu@master-node-dhiraj:~$ nano configmap-dev.yaml
ubuntu@master-node-dhiraj:~$ cat configmap-dev.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: site
data:
  APP_ENV: dev
  index.html: |
    <h1>Hello from DEV</h1>
    <p>Winter Arc Day 18</p>
ubuntu@master-node-dhiraj:~$ nano configmap-prod.yaml
ubuntu@master-node-dhiraj:~$ cat configmap-prod.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: site
data:
  APP_ENV: prod
  index.html: |
    <h1>Hello from PROD</h1>
    <p>Winter Arc Day 18</p>
ubuntu@master-node-dhiraj:~$ kubectl apply -f configmap-dev.yaml -n dev
configmap/site created
ubuntu@master-node-dhiraj:~$ kubectl apply -f configmap-prod.yaml -n prod
configmap/site created
ubuntu@master-node-dhiraj:~$ kubectl get configmap site -n dev
NAME   DATA   AGE
site   2      30s
ubuntu@master-node-dhiraj:~$ kubectl get configmap site -n prod
NAME   DATA   AGE
site   2      28s
```

04 / Secure Lab Secrets Creation (Task C)

Task C: Generated generic `db-secret` objects in both `dev` and `prod` namespaces using dummy literal values (`dev-pass` and `prod-pass`) to securely inject database credentials.

```
ubuntu@master-node-dhiraj:~$ kubectl create secret generic db-secret --from-literal=password=dev-pass -
n dev
secret/db-secret created
ubuntu@master-node-dhiraj:~$ kubectl create secret generic db-secret --from-literal=password=prod-pass
-n prod
secret/db-secret created
ubuntu@master-node-dhiraj:~$ kubectl get secret db-secret -n dev
NAME        TYPE     DATA   AGE
db-secret   Opaque   1      19s
ubuntu@master-node-dhiraj:~$ kubectl get secret db-secret -n prod
NAME        TYPE     DATA   AGE
db-secret   Opaque   1      19s
```

Security Best Practices

- Never commit plain-text or real production passwords to public repositories.
- Kubernetes Secrets store data as opaque base64-encoded strings by default.
- Always scope secrets to specific namespaces to maintain least-privilege access.

05 / Unified Workload Deployment (Task D)

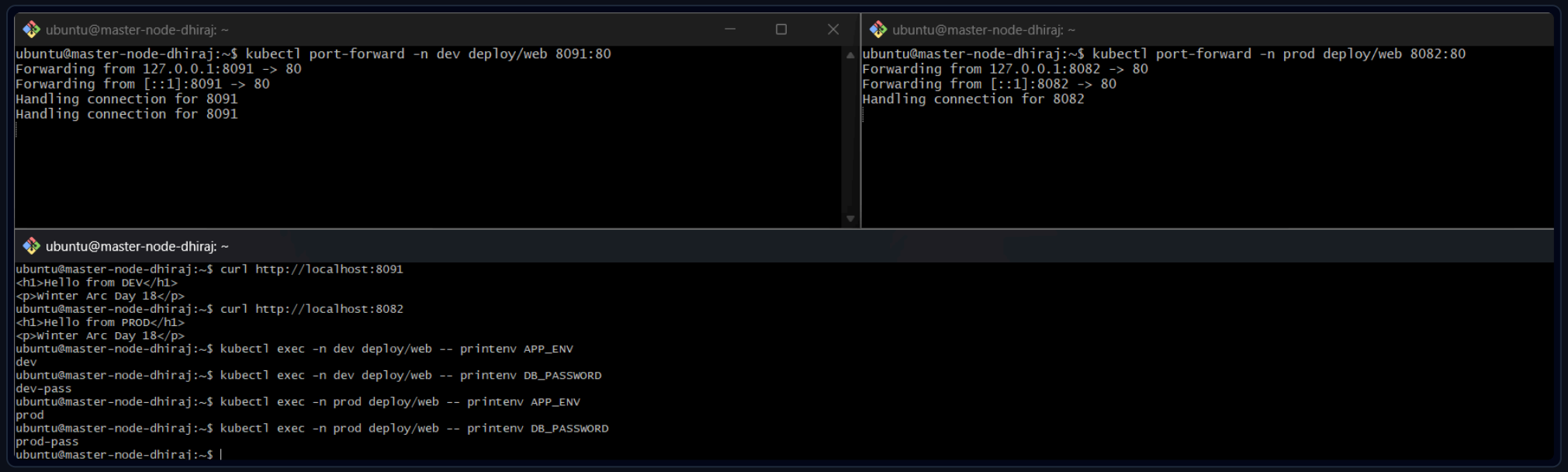
Task D: Applied `deployment.yaml` across both namespaces, binding ConfigMap values to environment variables and volume mounts (`/usr/share/nginx/html`), and referencing Secrets.

```
ubuntu@master-node-dhiraj:~$ nano deployment.yaml
ubuntu@master-node-dhiraj:~$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
        env:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: site
              key: APP_ENV
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: password
        volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
      volumes:
      - name: html
        configMap:
          name: site

ubuntu@master-node-dhiraj:~$ kubectl apply -f deployment.yaml -n dev
deployment.apps/web created
ubuntu@master-node-dhiraj:~$ kubectl apply -f deployment.yaml -n prod
deployment.apps/web created
ubuntu@master-node-dhiraj:~$ kubectl get pods -n dev
NAME                   READY   STATUS    RESTARTS   AGE
web-6849667f75-9zr68   1/1     Running   0          19s
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$
ubuntu@master-node-dhiraj:~$ kubectl get pods -n prod
NAME                   READY   STATUS    RESTARTS   AGE
web-6849667f75-l8qq6   1/1     Running   0          37s
ubuntu@master-node-dhiraj:~$
```

06 / Port-Forwarding & Environment Verification (Task E)

Task E: Forwarded ports (`8091` for dev, `8082` for prod), tested web outputs via `curl`, and verified injected environment variables using `kubectl exec`.



07 / Configuration Update & Rollout Restart (Task F)

Task F: Edited the dev ConfigMap (`dev-updated`), performed a rollout restart to update environment variables, and verified that container variables reflected the new changes.

```
ubuntu@master-node-dhiraj:~$ kubectl edit configmap site -n dev
configmap/site edited
ubuntu@master-node-dhiraj:~$ kubectl rollout restart deployment/web -n dev
deployment.apps/web restarted
ubuntu@master-node-dhiraj:~$ kubectl rollout status deployment/web -n dev
deployment "web" successfully rolled out
ubuntu@master-node-dhiraj:~$ kubectl exec -n dev deploy/web -- printenv APP_ENV
dev-updated
```

08 / Namespace Cleanup & Resource Deletion

Cleanup: Deleted the `dev` and `prod` namespaces after capturing all evidence screenshots, automatically terminating all contained pods, services, secrets, and configmaps.

```
ubuntu@master-node-dhiraj:~$ kubectl delete namespace dev
namespace "dev" deleted
ubuntu@master-node-dhiraj:~$ kubectl delete namespace prod
namespace "prod" deleted
ubuntu@master-node-dhiraj:~$
```

Cleanup Summary

- Executed `kubectl delete namespace dev` and `prod`.
- Ensured zero dangling resources or leftover lab clutter on the AWS EC2 cluster.
- Completed full challenge lifecycle successfully.

09 / Troubleshooting & Failure Analysis Matrix

Symptom / Error	What to Check	Resolution & Key Learning
CreateContainerConfigError	ConfigMap/Secret existence and key matching	Verify that the referenced ConfigMap or Secret exists in the exact same namespace and that key names match the manifest.
Resources appear missing	Namespace scoping	Check resources specifically within namespaces using `kubectl get all -n dev` and `-n prod`.
Wrong page appears	ConfigMap data & mount path	Inspect ConfigMap data keys and verify `/usr/share/nginx/html` volume mount configuration.
APP_ENV did not change	Environment variable lifecycle	Environment variables are loaded at startup. Trigger `kubectl rollout restart deployment/web` to apply updates.

10 / Deliverable & Review Criteria Checklist

Repository Structure (`day18-configmaps-secrets-namespaces/`)

- **configmap-dev.yaml & configmap-prod.yaml:** Environment-specific configuration maps.
- **deployment.yaml:** Unified deployment manifest utilizing ConfigMaps and Secrets.
- **README.md:** Detailed setup instructions, observations, and question answers.
- **evidence/screenshots:** Verified web pages, env values, and update behaviors.

Security & Review Standards

- **Security Compliance:** Dummy credentials used; zero production secrets committed to GitHub.
- **Core Competency:** Demonstrated namespace isolation, secret referencing, volume mounting, and environment variable update mechanics.

11 / Interview Readiness & Architecture Q&A

1. ConfigMap vs Secret?

- ConfigMaps store non-sensitive configuration data, while Secrets hold sensitive credentials encoded in base64.

2. Why same namespace?

- Kubernetes configuration objects are namespaced resources and cannot be accessed across different namespaces by default.

3. Env vars vs Mounted files?

- Env vars are injected at container start; mounted files dynamically sync from underlying volumes (unless using subPath).

4. Why base64 is not encryption?

- Base64 is mere encoding; anyone with read access can decode it. Secrets require encryption at rest and RBAC.